CHIMERA: Joint Horizontal and Vertical Pod Autoscaling with a Fast and a Slow Actuator

Adithyan C S S, Ajay M, and Rohith Pranov V
Computer Science and Engineering
Vellore Institute of Technology, Vellore, India

***Abstract*—Kubernetes can now change the CPU of a running pod in well under a second, while a new replica still takes tens of seconds to serve. Existing autoscalers use one of these actuators or run a separate loop for each. We present CHIMERA, a controller that plans replica count and CPU per pod together. It combines a linear-quadratic regulator whose state includes the replicas that have been ordered but are not yet serving, a cascade in which a five-second inner loop owns CPU and a fifteen-second outer loop owns replicas, and a capacity rule that prices the risk of missing a latency objective against the cost of CPU over conformal demand scenarios. Design choices were made on a development split and judged on data they did not see. In simulation, at equal CPU cost, CHIMERA has fewer objective violations than HPA, VPA, their combination, AAPA, a MagicScaler-style scaler and KOSMOS on 47 of 48 trace-by-baseline comparisons on a held-out set, and on all 117 on twenty real traces that no design step used: ten production serverless functions and ten web-server windows. The advantage persists, with a smaller margin, under latency laws the controller does not assume. An ablation shows that fast release of CPU, joint control and single ownership of each actuator carry the benefit, that a coupling penalty and a Lyapunov filter from the original design change no decision, and that the risk-priced capacity rule helps only when its latency model is roughly right. A Kubernetes operator runs the same control law. On a real cluster it resizes pods in place without restarts, and a small trace replay against the HPA (3 seeds) shows CHIMERA on or below the HPA's cost–violation frontier: 7 of 9 HPA runs were dominated by a CHIMERA run with the same seed, and no CHIMERA run was dominated.**

***Index Terms*—autoscaling, Kubernetes, in-place pod resize, serverless, optimal control, conformal prediction**

I. Introduction

A service on Kubernetes can be given more capacity in two ways: more pod replicas, or more CPU per pod. The two actuators differ by two orders of magnitude in speed. A new replica typically needs tens of seconds to be scheduled, started and made ready. Since in-place pod resize [1] (on by default from Kubernetes 1.33, stable in 1.35) the CPU of a running pod can be changed in well under a second without restarting the container.

The stock autoscalers do not exploit this. The Horizontal Pod Autoscaler (HPA) changes the replica count from average utilisation; the Vertical Pod Autoscaler (VPA) changes pod size from usage history. Run together on the same CPU signal they interfere, and neither models the fact that a replica it has just ordered is not serving yet. Predictive horizontal scalers such as AAPA [2] and MagicScaler [3] handle uncertainty but have only the slow actuator, so they must pay for forecast error with standing headroom. Systems that do use both actuators, such as KOSMOS [4], run two separately designed loops.

This paper asks how one controller should use a slow and a fast actuator on the same resource to hold a per-request latency objective at the lowest CPU cost. We present CHIMERA, which combines three ideas.

A delay-aware joint control law. Replicas and CPU per pod are driven by one linear-quadratic regulator whose state includes the replicas that have been ordered but are not yet serving, so the same capacity is never ordered twice.

A cascade that gives each actuator one owner. A 5 s inner loop owns CPU: it raises CPU as soon as a surge is statistically visible and returns it shortly after the surge passes. A 15 s outer loop owns replicas and changes them only when CPU resizing cannot absorb the change.

Risk-priced capacity. The amount of capacity is chosen by minimising expected violations of the objective plus a price per core, over demand scenarios drawn from conformal forecast errors.

We do not claim to be the first to resize in place before scaling out (Themis [5] does so for inference serving), to optimise both axes jointly (Ship et al. [6] pose it as a fluid optimal-control problem), to trade cost against violation risk (MagicScaler), or to use conformal calibration in an autoscaler (BACC [7], for the horizontal axis). Our contributions are:

- the combination above for a sub-second and a tens-of-seconds actuator, with an explicit model of replicas in flight (Section III);
- an evaluation protocol that separates design data from verdict data and compares controllers at equal cost (Section IV);
- results on a held-out set, on twenty real traces no design step used, and under latency laws the controller does not assume (Sections V and VI);
- an account of which parts of the design carry the benefit, including two mechanisms that turned out to change no decision and one that helps only under its own assumptions;
- a Kubernetes operator, validated on Kubernetes 1.33 and compared with the HPA by trace replay (Section VII).

II. Background and Related Work

In-place resize. KEP-1287 [1] lets the kubelet change a running container's CPU and memory through the `pods/resize` subresource. For CPU this is a cgroup update and takes effect without a restart. Hsieh and Chou [8] study the feature for serverless cold starts; Sponge [9] uses it to follow changing objectives in inference serving.

Joint horizontal and vertical scaling. Autopilot [10] scales both axes at Google from usage history, with slack and out-of-memory rate as objectives. SHOWAR [11] sizes pods from the variance of historical usage and scales replicas with a controller on kernel metrics. KOSMOS [4], the closest system to ours, resizes containers with control-theoretic planners on response time and adds or removes replicas with a heuristic controller. Themis [5] absorbs a surge with in-place vertical scaling and moves to horizontal scaling once the workload settles; it chooses configurations by dynamic programming over profiled latency, forecasts with an LSTM without prediction intervals, and handles replica start-up through its transition procedure, not through a model of replicas in flight. Ship et al. [6] derive resources per replica and replica counts from a fluid approximation of a queueing network solved as a separated continuous linear program; their model has no replica start-up delay and takes demand rates as given. Autothrottle [12] uses a bi-level structure for CPU alone: a learned application-level controller sets throttle targets that per-service controllers meet. CHIMERA differs from these in planning both actuators from one target with an explicit state for in-flight replicas, in closing the loop every five seconds, and in sizing capacity by priced risk.

Predictive and uncertainty-aware scaling. MagicScaler [3] forecasts demand with a multi-scale attentive Gaussian process and chooses the number of instances with a loss that trades running cost against the risk of violating quality of service; this is the nearest precedent for our capacity rule. AAPA [2] classifies serverless workloads into four archetypes and applies a target and cooldown per archetype, widened by calibrated confidence. BACC [7] wraps a forecaster in adaptive conformal inference and steers horizontal provisioning with a controller on a violation budget. FIRM [13] localises the microservice responsible for a violation and reprovisions it. All of these act on one axis.

III. Design

Fig. 1 gives the structure. Every 15 s the outer loop runs the steps of Sections III-A–III-D; every 5 s the inner loop runs Section III-E.

A. Forecast

A gradient-boosted quantile model, conformalised on a held-back window [14], gives a 90 % interval for next-minute demand. Because a learned model has nothing to say about a regime it has not seen, a second, model-free margin is kept beside it: the last observed value plus a split-conformal quantile of recent one-step changes. An ADWIN detector [15] on the forecast error discards that margin's history when the regime changes. The operator, which has no training history for a newly attached service, uses the model-free construction around an exponentially weighted mean.

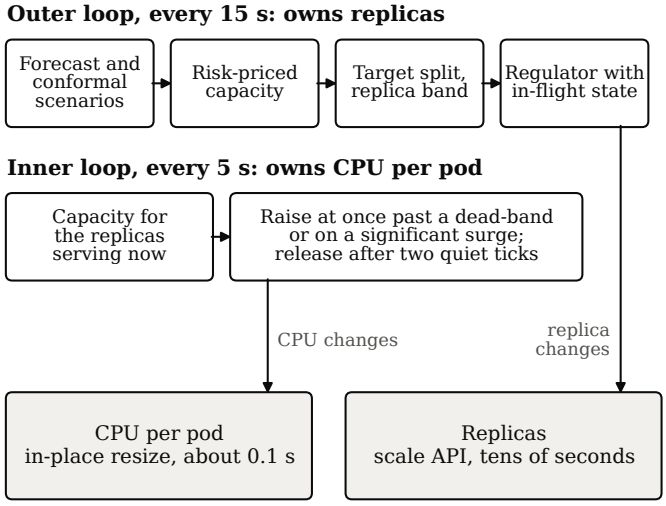


Fig. 1. The two loops and the actuator each owns.

B. Risk-priced capacity

Let u be utilisation and suppose a request misses the latency objective with probability $p(u) = \exp(-k(1-u))$ below saturation, where k is the ratio of the objective to the mean service time, estimated online. Given demand scenarios $L_1, \dots, L_n$ (the current load, the current load plus each recent signed one-step change, and the forecast upper bound), total capacity is

$$c^* = \arg\min_{c} \; \frac{1}{n} \sum_{i=1}^{n} L_i \, p\!\left(\frac{L_i}{c}\right) + \lambda c, \qquad \lambda = p'(u_0)\, u_0^2. \quad (1)$$

The price λ is fixed by one knob u_0: with it, a demand known with certainty gives $c^* = L/u_0$, so u_0 reads as an ordinary target utilisation. Under uncertainty the convexity of p decides how much headroom is worth buying. The objective is not convex (violation probability saturates near full utilisation), so it is minimised on a grid and refined, over capacities that keep the expected load below saturation.

C. Target split and replica band

Capacity is split into a target $x^* = [v^*, h^*]$ (CPU per pod, replicas) with $h^* v_{\max} \geq c^*$. Between $\lceil c^*/(0.85\, v_{\max}) \rceil$ and $\lfloor c^*/v_{\min} \rfloor$ replicas, any replica count can carry c^* by resizing CPU alone, while every added replica spends its boot time allocated but not serving. Inside this band h^* is left at the current count. v^* is sized for the replicas that are serving now, so the fast actuator covers the gap while ordered replicas boot and hands the capacity back as they arrive.

D. Delay-augmented regulator

The tracking error is augmented with the replicas in flight:

$$e = \left[\, v - v^{*},\; h_{\text{serving}} - h^{*},\; \Delta h_{t-1}, \ldots, \Delta h_{t-d} \,\right]^{\top}, \quad u = -Ke, \tag{2}$$

with K from the discrete algebraic Riccati equation for $Q = \mathrm{diag}(30, 10)$ and $R = \mathrm{diag}(0.05, 1)$, and $d = \lfloor \tau / \Delta t \rfloor$ slots for a boot time τ and control period Δt (one slot for 20 s and 15 s). Only scale-ups enter the buffer; a removed replica stops serving at once. The action is clipped to the resource bounds and a total capacity budget. Closed-loop stability follows from the Riccati solution P: $V = e^{\top} P e$ decreases along $u = -Ke$.

On a real cluster boot time is not a constant. The operator therefore reconciles the buffer with what the cluster reports, taking the replicas in flight to be the desired count minus the ready count, before each step (Section VII describes the failure this prevents).

E. Inner loop

Between outer steps only CPU moves. Every 5 s the inner loop recomputes the capacity of Section III-B for the serving replicas. It raises CPU per pod at once if the need exceeds the allocation by a relative dead-band (30 %), or if a Poisson test on a short arrival window shows a significant surge, and lowers it once the need has stayed 15 % below the allocation for two consecutive ticks. Acquiring CPU takes about 0.1 s, so there is no reason to hold it through a long cooldown. The dead-band bounds how often pods are resized.

F. Implementation

The controller is about 350 lines of Python; one control step takes about 0.3 ms. The Kubernetes operator watches a custom resource, reads CPU usage and optionally request rate from Prometheus, resizes CPU through `pods/resize`, changes replicas through the Deployment scale subresource, and writes its decision to the resource's status.

IV. Evaluation Method

A. Simulator

We extend the discrete-event simulator released with AAPA [2] to two dimensions: pods have a CPU size, a replica takes 20 s to boot, and an in-place resize takes 0.1 s. Requests have exponentially distributed CPU service times with mean 0.1 s and are slowed by a factor $1/(1-u)$ at utilisation u; the latency objective is 0.5 s. Section VI-B replaces both the distribution and the slowdown law.

B. Workloads and splits

Table I lists the trace sets. Every design choice was made on the dev set. All sets use an unscored 20-minute warm-up so that no controller is scored on its cold start. The Azure excerpts appear in both the dev and the held-out set, in different windows; the two external sets exist to remove that dependence. Their functions and windows were fixed by rule before any controller ran on them. Request rates are rescaled so that the busiest minute of a window fits what the horizontal-only baselines can serve at their replica limit.

TABLE I
Trace sets. Each trace is 100 minutes; minutes 70–100 are scored.

Set	Traces	Role
Dev	5 Azure Functions 2019 excerpts [16] (minutes 40–70), 4 synthetic shapes; 3 seeds	design
Held-out	the same 5 Azure excerpts (minutes 70–100), 4 other synthetic shapes; 10 new seeds	verdict
Serverless	10 production functions, Huawei private cloud 2023 [17]	verdict
Web	6 NASA and 4 ClarkNet web-server windows, 1995 [18]	verdict

C. Baselines

HPA; VPA with five fixed replicas; HPA and VPA together; AAPA [2]; a scaler in the style of MagicScaler [3] (our implementation of the idea, not the authors' predictor or code); and KOSMOS [4], reimplemented from the paper with its gains tuned on the dev set. Every controller has the same resource bounds and starts from the same allocation.

D. Comparison at equal CPU

Controllers at their default settings sit at different points of the cost–violation trade-off, so a lower violation rate may only mean more CPU was spent. Every controller is therefore swept over its own conservatism setting (target utilisation; for KOSMOS its response-time setpoint), and each baseline's violation rate is interpolated on its own frontier at CHIMERA's CPU cost, trace by trace, wherever the two cost ranges overlap. Significance is tested on one gap per trace and seed (the same request stream) with a one-sample Wilcoxon signed-rank test, Holm-corrected across baselines.

V. Results on the Held-Out Set

The main comparison is 3,870 runs (43 configurations, 9 traces, 10 seeds); the frontier sweep is 4,095 runs (5 seeds).

A. Equal CPU

Table II gives, for each baseline, the number of traces on which CHIMERA has fewer violations at the same CPU cost and the mean difference. Counting per-trace frontier comparisons, CHIMERA wins 47 of 48. The exception is the MagicScaler-style scaler on one Azure trace (a high, flat, noisy load), by 0.66 points. VPA is compared on fewer traces because its cost range, fixed by its five replicas, does not always overlap CHIMERA's.

B. Default settings

Table III shows each controller at its default setting. Against AAPA on the Azure traces CHIMERA has 0.99 points fewer violations ($p = 0.003$) for a CPU difference of $+14.7$ core-minutes that is not significant ($p = 0.086$); on the synthetic shapes it has 1.39 points fewer violations with 30 % less CPU. Two things in the table do not favour CHIMERA. KOSMOS has fewer violations on the synthetic shapes (3.42 % against 5.63 %) at 1.8 times the CPU; the equal-CPU comparison is

TABLE II
HELD-OUT SET AT EQUAL CPU. GAP = BASELINE MINUS CHIMERA, IN PERCENTAGE POINTS OF VIOLATION RATE. ALL $p < 0.001$ (WILCOXON, HOLM).

	Azure		Synthetic	
Baseline	won	gap	won	gap
AAPA	5/5	+1.68	4/4	+5.28
HPA	5/5	+4.71	4/4	+8.04
HPA+VPA	5/5	+4.26	4/4	+6.27
KOSMOS	5/5	+5.65	4/4	+6.00
MagicScaler-style	4/5	+4.72	4/4	+7.31
VPA	3/3	+2.81	3/3	+9.69

TABLE III
HELD-OUT SET, DEFAULT SETTINGS. VIOL. = SHARE OF REQUESTS OVER THE OBJECTIVE; CPU IN CORE-MINUTES PER 30-MINUTE RUN; ACT. = SCALING ACTIONS PER RUN.

	Azure		Synthetic		
Controller	Viol.	CPU	Viol.	CPU	Act.
CHIMERA	5.93 %	251.5	5.63 %	71.8	32.5
AAPA	6.92 %	236.8	7.02 %	101.9	6.7
KOSMOS	8.71 %	326.7	3.42 %	132.1	274.8
HPA+VPA	14.06 %	194.8	13.29 %	65.2	19.2
HPA	15.60 %	190.8	15.13 %	63.7	14.5
MagicScaler-style	16.78 %	181.1	14.41 %	63.2	16.1
VPA	19.40 %	169.2	15.16 %	78.7	10.8

what separates them. And CHIMERA has the lower seed-mean violation rate than AAPA on only 3 of the 5 Azure traces.

C. Where the benefit comes from

Table IV removes or re-adds one mechanism at a time. Rows are single operating points: a row with fewer violations and more CPU is a different point on the trade-off, not a better controller. Read that way, fast release is the largest effect: without it CPU nearly doubles on the Azure traces and is 2.6 times higher on the synthetic shapes. Joint control saves 7 % CPU on the Azure traces and 37 % on the synthetic shapes against horizontal-only control, with 0.5 pod changes per run instead of 12. Single ownership of CPU and the dead-band each remove more than half of the resizes that would otherwise occur, and the replica band cuts pod changes from 8.6 to 0.5 per run.

The last three rows are negative results. The design began with a penalty on opposing moves of the two actuators and a Lyapunov filter on the action; re-adding either, or a guardrail against scaling in while replicas boot, leaves every one of the 90 runs identical. Once each actuator has a single owner and replicas move rarely, opposing moves do not arise, and the filter enforces a decrease the regulator already guarantees. The reported controller does not run them.

Without in-place resize (a CPU change then needs a rolling restart) CHIMERA's violation rate on the synthetic shapes rises from 5.63 % to 7.91 %: the design depends on the fast actuator.

TABLE IV
ABLATION ON THE HELD-OUT SET. RS. = IN-PLACE RESIZES PER RUN; PODS = REPLICA CHANGES PER RUN.

	Azure		Synthetic			
Variant	Viol.	CPU	Viol.	CPU	Rs.	Pods
Full controller	5.93	251.5	5.63	71.8	32.5	0.5
w/o fast release	2.24	480.1	2.44	185.6	1.0	0.5
horizontal only	4.80	271.7	3.75	114.8	12.0	12.0
w/o inner loop	4.92	274.6	5.01	81.2	37.2	0.4
CPU not single-owned	5.12	267.4	4.90	75.7	79.7	0.5
w/o dead-band	4.96	269.1	4.80	75.8	84.1	0.5
w/o replica band	5.88	256.2	5.61	74.3	43.3	8.6
w/o CQR	6.21	242.6	5.62	69.8	37.8	0.4
+ coupling penalty	5.93	251.5	5.63	71.8	32.5	0.5
+ Lyapunov filter	5.93	251.5	5.63	71.8	32.5	0.5
+ scale-in guardrail	5.93	251.5	5.63	71.8	32.5	0.5

TABLE V
TWENTY EXTERNAL REAL TRACES AT EQUAL CPU. GAP IN PERCENTAGE POINTS; ALL $p < 0.001$.

	Serverless		Web	
Baseline	won	gap	won	gap
AAPA	10/10	+2.02	10/10	+2.00
HPA	10/10	+3.99	10/10	+4.53
HPA+VPA	10/10	+2.72	10/10	+3.89
KOSMOS	10/10	+2.38	10/10	+1.27
MagicScaler-style	10/10	+4.37	10/10	+4.66
VPA	9/9	+1.77	10/10	+2.20

VI. DOES THE RESULT GENERALISE?

A. Real traces no design step used

The Azure excerpts are few and were visible, in an earlier window, during design. We therefore ran the unchanged controllers on two further sets: ten production serverless functions from the Huawei private-cloud trace [17], and ten windows of the NASA and ClarkNet web-server logs [18] (900 main runs and 3,150 sweep runs each). Table V and Fig. 2 give the results.

CHIMERA wins all 117 per-trace frontier comparisons (57 serverless, 60 web). At default settings on the serverless traces it has 6.06 % violations at 251 core-minutes against AAPA's 9.18 % at 230; on the web traces 6.18 % at 190 against 7.45 % at 194. KOSMOS is the closest competitor. At its default setting it matches CHIMERA on the serverless traces (6.16 %) and beats it on the web traces (4.22 %), for 45 % and 43 % more CPU and about ten times as many resizes.

A first version of the web experiment scaled the request rates too high: the horizontal-only baselines sat at their replica limit on every ClarkNet window and CHIMERA "won" 56 of 58 comparisons for the wrong reason. We discarded it, rescaled so that the busiest minute fits the baselines' limit, and report only the re-run.

B. A latency model the controller does not assume

The capacity rule (1) assumes the same latency law the simulator generates. To test whether the result depends on that

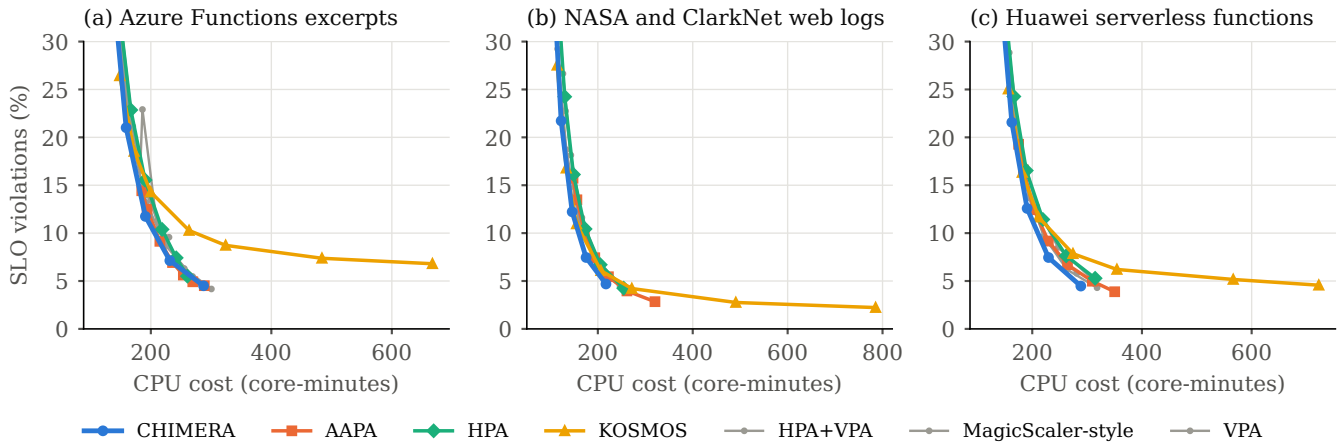


Fig. 2. Cost–violation frontiers on the three verdict sets. Each line sweeps one controller over its conservatism setting; down and to the left is better.

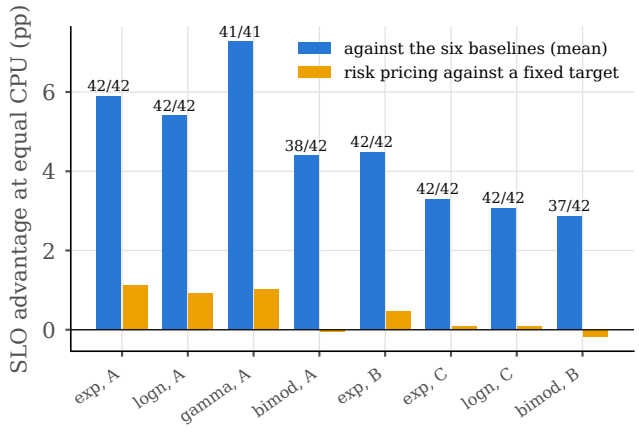


Fig. 3. Mean advantage at equal CPU under eight latency models. Bar labels give the comparisons won out of those possible. Service times: exponential, lognormal, gamma, bimodal. Slowdown laws: A $= 1/(1-u)$, B $= 1 + u/(2(1-u))$, C $= 1/(1-u^4)$. The first model is the one the controller assumes.

match, we re-ran the held-out sweep under three other service-time distributions (lognormal, gamma, bimodal; same mean) and two other slowdown laws, without re-tuning anything (Fig. 3; this experiment used the previous 10 % dead-band, which Table IV shows moves the frontier little).

The advantage over the baselines survives in every scenario, on every trace in six of eight, with a mean gap between 2.9 and 7.3 points. With bimodal service times CHIMERA loses to AAPA at equal CPU on four traces, by 0.1 to 0.7 points. The second series isolates the capacity rule by comparing the controller with and without it: it is worth about one point while the slowdown law matches its assumption, nothing under a hockey-stick law, and slightly less than nothing with bimodal service times. On the external traces it is worth 0.2 points. The structural parts of the design carry the robust part of the result; the capacity rule is a refinement that needs its latency model to be roughly right.

VII. On a Real Cluster

A. Functional validation, and what it found

On Kubernetes 1.33 (a local kind cluster, and one cloud instance running k3s) the operator raised the CPU of running pods in place under load with no container restart, added a replica once CPU alone could not cover the need, returned CPU when the new replica was serving, and released everything when the load stopped.

The first run also removed a replica 15 s after adding it. The controller's in-flight buffer assumed the configured boot time of 20 s; the real pod was ready within one control period, so it was counted both as serving and as in flight. The simulator, whose boot time is a constant, could not show this. The operator now reconciles the buffer with the cluster before each step. Later runs exposed four further operator faults of the same kind (sizing below observed demand on a cold start, a rate-based demand estimate below measured CPU, a configured cost per request overwritten by online learning, and scale-in directly after scale-out); each is fixed and covered by a regression test. We report this because it is the reason to run on a cluster at all: it tests the assumptions the simulator bakes in.

B. Trace replay against the HPA

An open-loop load generator replayed a 24-minute demand trace (12 minutes of an Azure trace, then a flash crowd to five times the mean) against a CPU-bound service costing 61 ms of CPU per request, with a 0.3 s latency objective (five times the service time, as in the simulator). Each controller started from the same allocation with three unscored lead-in minutes, and both sides could reach the same ten cores. We ran the HPA at three utilisation targets and CHIMERA at 3 settings, so that each has a cost frontier, with 3 seeds each (Table VI, Fig. 4). CHIMERA used a request-rate signal; no run restarted a container.

Of the 9 HPA runs, 7 were dominated by a CHIMERA run with the same seed: one that used no more CPU and missed the objective less often. Of the 9 CHIMERA runs, 0

TABLE VI
LIVE REPLAY, MEAN OF 3 SEEDS. VIOL. = REQUESTS OVER 0.3 S (%), WITH THE RANGE OVER SEEDS; >1 S = REQUESTS OVER ONE SECOND (%); CPU IN CORE-MINUTES.

Controller	Viol.	Range	>1 s	CPU
HPA, 70 % target	48.9	44.1–55.2	39.3	85.3
HPA, 50 % target	23.5	21.9–26.4	14.9	102.4
HPA, 35 % target	16.3	11.2–22.5	5.4	124.3
CHIMERA, 0.46 (default)	32.1	26.2–43.1	20.9	80.9
CHIMERA, 0.35	14.6	12.1–18.7	7.1	93.6
CHIMERA, 0.25	7.0	6.6–7.7	3.2	108.1

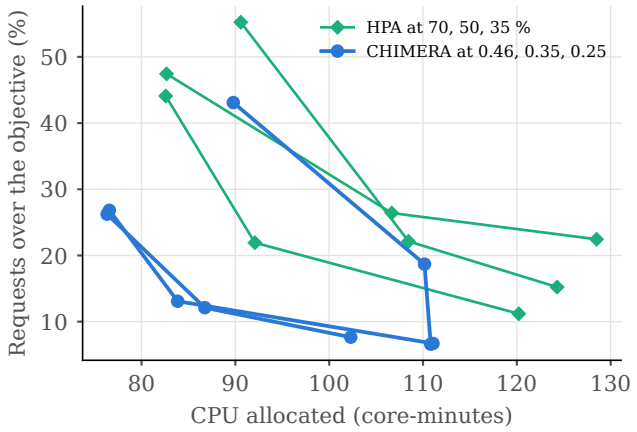


Fig. 4. Live replay: share of requests over the objective against CPU allocated. One line per seed joins the settings of each controller; down and to the left is better.

were dominated by an HPA run. For the 6 CHIMERA runs whose cost lay inside the range spanned by the HPA settings, interpolating the HPA's violation rate at the same cost gave CHIMERA fewer violations in 5, with a mean gap of +11.9 points. At setting 0.35 CHIMERA averaged 14.6 % violations for 93.6 core-minutes, against 16.3 % for 124.3 with the HPA at its most generous setting. One CHIMERA run (seed 2, setting 0.25) was repeated. In the first attempt the service was slow whatever it was given (median latency 0.31 s with four to ten cores allocated for at most three cores of demand, and 20540 requests issued where the other runs issued between 22 000 and 22 900), which points to interference on the host and not to a control decision; it had 50.9 % violations. Counting that attempt instead of the repeat, 7 of 9 HPA runs are dominated and 1 of 9 CHIMERA runs. Both attempts are in the repository. With 3 seeds, one service and one trace this is indicative, not a significance result, and the spread between seeds is large. Every controller missed the objective far more often than in simulation: a 61 ms request on a pod limited to half a core exceeds a single CFS quota period, so the simulator's latency law is kinder than this service.

VIII. DISCUSSION AND LIMITATIONS

What carries the result. The fast actuator and single ownership. Returning CPU within two ticks of a surge passing, instead of holding it through a cooldown, accounts for about half of the CPU a controller with only slow release would spend. Giving CPU to one loop and replicas to another, with a band in which replicas do not move, removes the interference that the original design tried to penalise away.

What does not. Risk-priced capacity is worth about a point when the latency law matches its assumption and nothing when it does not. It should be learned from measured latency, not derived from a model, before it is relied on.

KOSMOS is close. An earlier version of this evaluation used KOSMOS gains tuned before a simulator fix, with which its vertical loop oscillated across its whole range, and showed a nineteen-point advantage. With gains re-tuned on the dev set the advantage is 5.8 points on the held-out set, 2.4 on the serverless traces and 1.3 on the web traces, and at default settings KOSMOS matches or beats CHIMERA's violation rate on three of the four trace groups at substantially higher CPU.

Limitations. (1) The quantitative claims rest on simulation; the live experiment is small. (2) The Azure excerpts were visible, in a different window, during design; the serverless and web traces were not, but they are twenty 100-minute windows from three sites. (3) The held-out set was scored twice, once for the previous controller version and once for the final one; the final change was chosen on the dev set by a rule fixed beforehand. (4) KOSMOS and the MagicScaler-style scaler are our implementations from the papers; VPA runs with five fixed replicas. (5) In the simulator boot time and resize latency are constants and the slowdown law is analytic; Section VII shows a real service behaving worse at the same nominal utilisation. (6) CHIMERA makes about 31 in-place resizes per half hour per workload, against about 7 pod changes for AAPA; the load this places on the API server and kubelet at fleet scale is not measured. (7) Only CPU is scaled; shrinking memory in place can fail.

IX. CONCLUSION

A controller that treats replica count and CPU per pod as one problem, gives each actuator a single owner, and uses the fast one to cover the slow one's delay holds a per-request latency objective with fewer violations than HPA, VPA, their combination, AAPA, a MagicScaler-style scaler and KOSMOS at the same CPU cost: on 47 of 48 comparisons on a held-out set and on all 117 on twenty real traces no design step used. The advantage persists under latency laws the controller does not assume, with a smaller margin. On a real cluster a small trace-replay experiment shows CHIMERA on or below the HPA's cost–violation frontier: 7 of 9 HPA runs were dominated by a CHIMERA run with the same seed, and no CHIMERA run was dominated. A larger live study, with more seeds, more services and a simulator whose latency law is fitted to measurements, is the main work that remains.

REFERENCES

[1] Kubernetes Enhancement Proposal 1287, "In-place update of pod resources"; see also "Kubernetes v1.35: In-place pod resize graduates to stable," Kubernetes Blog, Dec. 2025.

[2] G. Zhang, S. Vippagunta, R. Nandagopal, S. Raman, J. Xu, M. Pfeiffer, S. Chatterjee, Z. Tan, W. Guo, and H. Jiang, "AAPA: An archetype-aware predictive autoscaler with uncertainty quantification for serverless workloads on Kubernetes," arXiv:2507.05653, 2025.
[3] Z. Pan *et al.*, "MagicScaler: Uncertainty-aware, predictive autoscaling," *Proc. VLDB Endow.*, vol. 16, no. 12, pp. 3808–3821, 2023.
[4] L. Baresi, D. Y. X. Hu, G. Quattrocchi, and L. Terracciano, "KOSMOS: Vertical and horizontal resource autoscaling for Kubernetes," in *Proc. ICSOC*, LNCS, 2021, pp. 821–829.
[5] K. Razavi, M. Salmani, M. Mühlhäuser, B. Koldehofe, and L. Wang, "A tale of two scales: Reconciling horizontal and vertical scaling for inference serving systems," arXiv:2407.14843, 2024.
[6] H. Ship, E. Shindin, C. Wang, D. Arroyo, and A. Tantawi, "Optimizing simultaneous autoscaling for serverless cloud computing," in *Proc. IEEE CLOUD*, 2024; arXiv:2310.19013.
[7] F. Liu, G. Li, B. Farkiani, and P. Crowley, "BACC: Budget-aware calibration and control for horizontal autoscaling," arXiv:2606.20575, 2026.
[8] V. Hsieh and J. Chou, "Towards serverless optimization with in-place scaling," arXiv:2311.09526, 2023.
[9] K. Razavi, S. Ghafouri, M. Mühlhäuser, P. Jamshidi, and L. Wang, "Sponge: Inference serving with dynamic SLOs using in-place vertical scaling," in *Proc. EuroMLSys*, 2024.
[10] K. Rzadca *et al.*, "Autopilot: Workload autoscaling at Google," in *Proc. EuroSys*, 2020.
[11] A. F. Baarzi and G. Kesidis, "SHOWAR: Right-sizing and efficient scheduling of microservices," in *Proc. ACM SoCC*, 2021, pp. 427–441.
[12] Z. Wang, P. Li, C.-J. M. Liang, F. Wu, and F. Y. Yan, "Autothrottle: A practical bi-level approach to resource management for SLO-targeted microservices," in *Proc. USENIX NSDI*, 2024.
[13] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, "FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices," in *Proc. USENIX OSDI*, 2020.
[14] Y. Romano, E. Patterson, and E. Candès, "Conformalized quantile regression," in *Proc. NeurIPS*, 2019.
[15] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM SDM*, 2007.
[16] M. Shahrad *et al.*, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX ATC*, 2020.
[17] A. Joosen *et al.*, "How does it function? Characterizing long-term trends in production serverless workloads," in *Proc. ACM SoCC*, 2023; data: github.com/sir-lab/data-release.
[18] Internet Traffic Archive, NASA-HTTP and ClarkNet-HTTP traces, Lawrence Berkeley National Laboratory, https://ita.ee.lbl.gov/html/traces.html.